

Generative AI vs Agentic AI

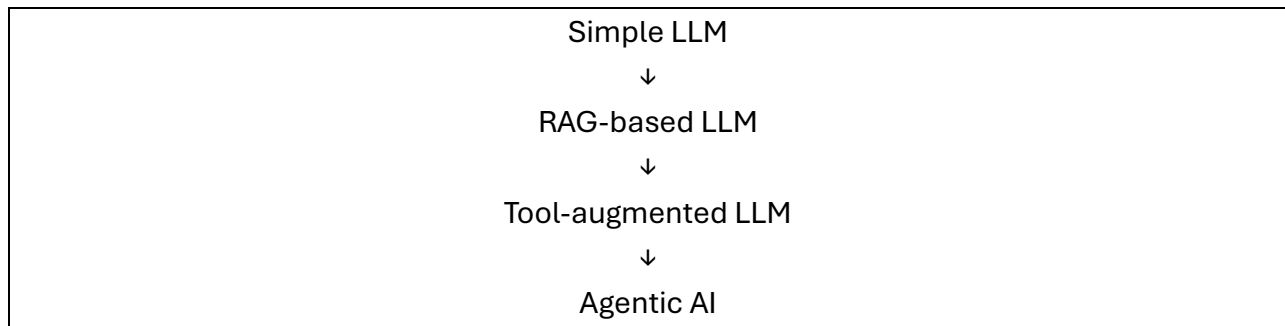
It is an important transition from **Generative AI** to **Agentic AI** and eventually to **LangGraph**. You have already learned about **LLMs, RAG, tools, tool calling, and agents**. This topic puts those concepts into one bigger picture.

The easiest way to understand it is:

Generative AI creates an answer. Agentic AI works toward a goal.

❖ Introduction:

There are roughly four levels of AI applications we'll see in this lesson:



Each level gives the AI more capabilities.

For example, imagine asking:

"Plan a 5-day trip to Islamabad."

A simple LLM can **generate a plan**.

A RAG system can use your travel documents or a knowledge base.

A tool-augmented system can use:

- Weather API
- Maps
- Hotel API

An agent can decide **which information it needs, which tools to use, and in what order**.

That's the progression this video demonstrates.

❖ What is Generative AI?

Generative AI is AI that can create new content based on input or instruction.

It can generate:

-  Text
-  Images
-  Audio
-  Video

-  Code

❖ Example

You ask:

"Write a Python function to calculate factorial."

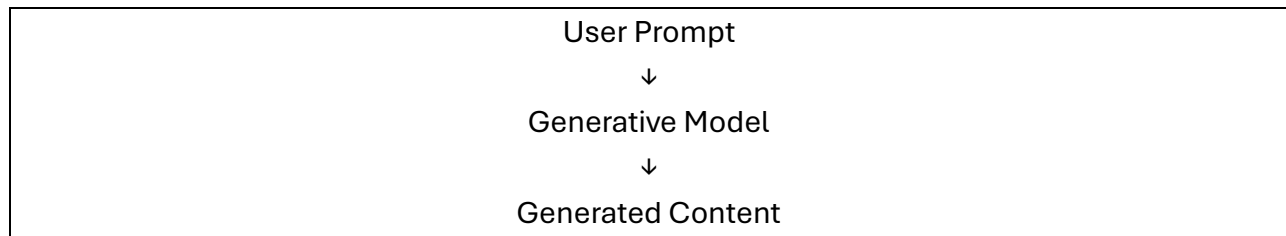
Generative AI produces:

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)
```

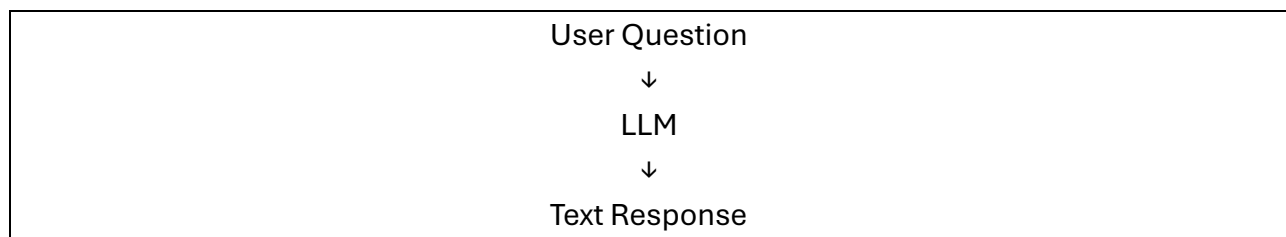
The AI **generated something new** based on your instruction.

❖ How Does Generative AI Work?

A simplified view:



For an LLM:



The model receives an input and predicts/generates an appropriate output.

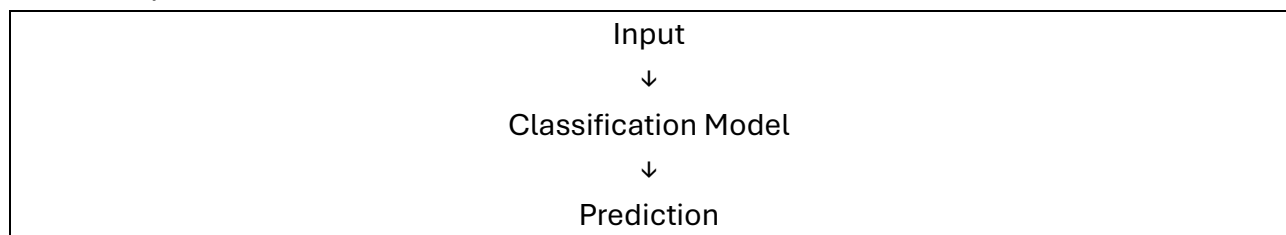
❖ Generative AI vs Traditional AI

This distinction is important.

❖ Traditional AI

Traditional AI generally performs a specific predefined task.

For example:



Suppose you give an email:

"Congratulations! You've won a prize."

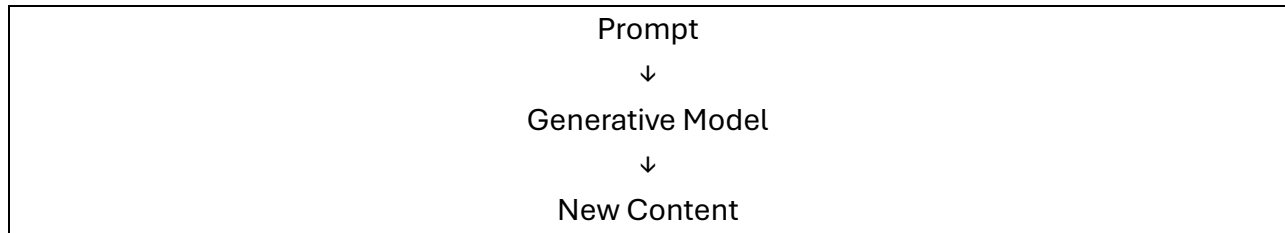
A spam classifier might output:

Spam

It is primarily **classifying or predicting**.

❖ **Generative AI**

Generative AI can produce new content.



For example:

"Write a professional email asking for leave."

The model generates the actual email.

❖ **Simple Comparison**

Traditional AI	Generative AI
Predicts/classifies	Creates content
Usually task-specific	More flexible
Often predefined output	Can produce varied output
Example: spam detection	Example: ChatGPT

❖ **12:17 – Applications of Generative AI**

Generative AI has many applications.



Text Generation

- Emails
- Articles
- Summaries
- Reports
- Stories



Code Generation

- Writing code
- Explaining code
- Debugging
- Test generation

Image Generation

- Marketing images
- Illustrations
- Concept art

Audio

- Voice generation
- Music generation
- Voice conversion

Video

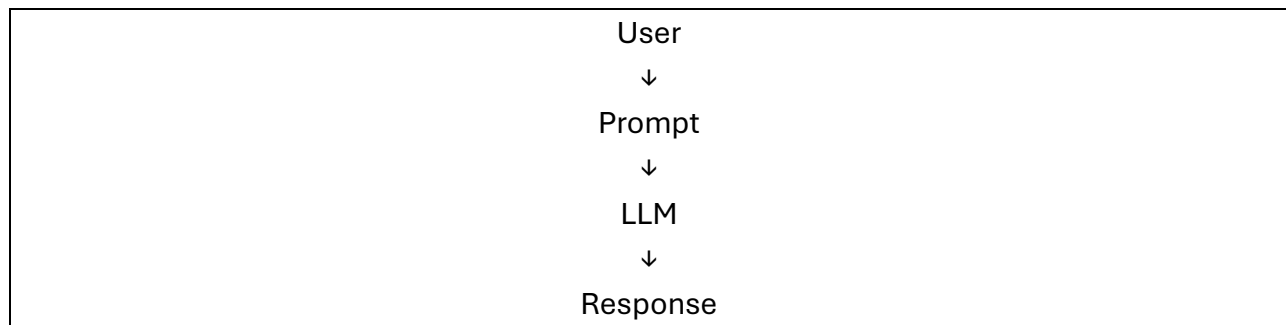
- AI-generated videos
- Animation
- Video editing

❖ Chatbot 1

Now the video starts building increasingly capable chatbots.

The first chatbot is essentially a **basic LLM chatbot**.

Its architecture is:



Example

User:

"What is Pakistan's capital?"

LLM:

"Islamabad."

Simple.

❖ Problems with Chatbot 1

The basic chatbot has important limitations.

Problem 1 — No Private Knowledge

Suppose you ask:

"What is written in my company's employee handbook?"
The LLM doesn't automatically know your handbook.

❌ Problem 2 — Knowledge Cutoff

The model may not know recent information.
For example:

"What happened today?"

A normal LLM may not have access to today's events.

❌ Problem 3 — Hallucination

The model can sometimes generate information that sounds convincing but is incorrect.

❌ Problem 4 — No External Tools

A basic LLM cannot automatically:

- Query your database
- Call your company's API
- Check live exchange rates
- Search current information

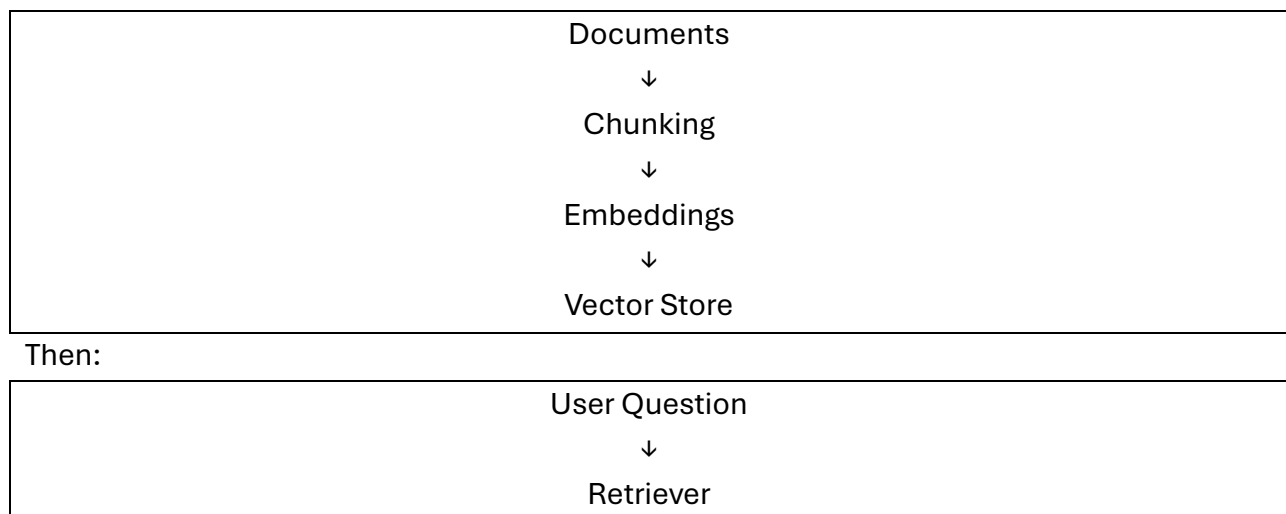
❖ Chatbot 2: RAG-Based

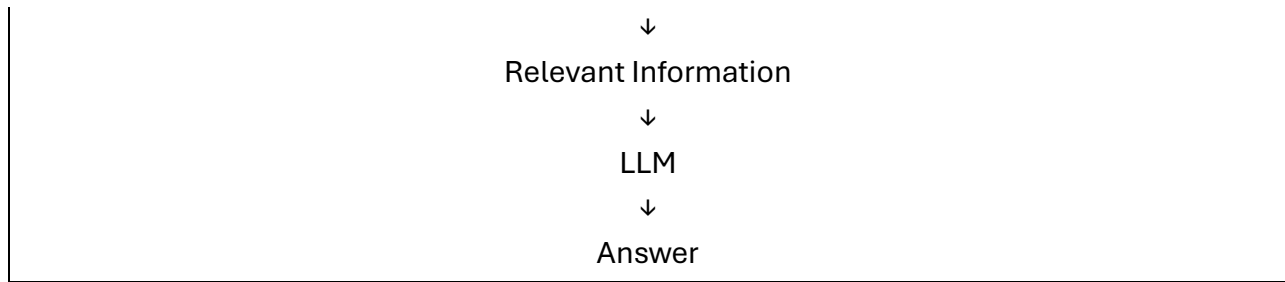
The next improvement is:

RAG — Retrieval-Augmented Generation

Instead of asking the LLM to answer only from its internal knowledge, we give it relevant external information.

❖ Architecture





Example

Suppose you have:

University Handbook.pdf

User asks:

"What is the minimum attendance requirement?"

RAG searches the handbook and retrieves the relevant section.

The LLM then answers using that information.

❖ Why RAG Is Better

It gives the LLM access to:

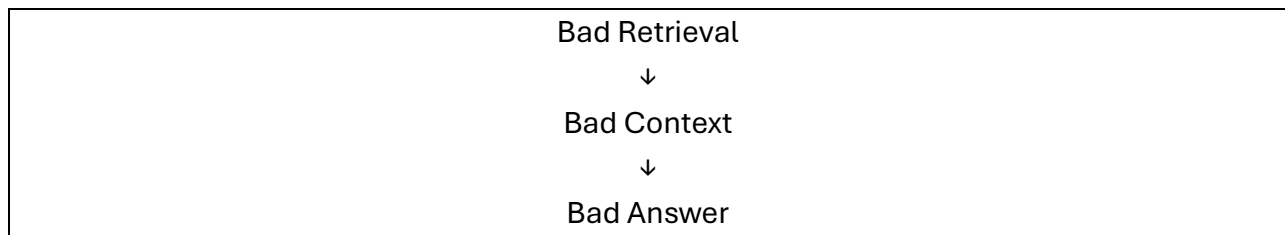
- Private documents
- Company knowledge
- Updated information
- Domain-specific information

❖ Problems with Chatbot 2

RAG solves some problems, but not everything.

❌ Problem 1 — Retrieval Can Fail

If the retriever finds the wrong chunks, the LLM receives bad context.



❌ Problem 2 — RAG Only Retrieves

RAG is primarily designed to **find relevant information**.

It doesn't automatically decide:

"Should I search the database?"

or

"Should I call an API?"

❌ Problem 3 — Limited Actions

Suppose the user says:

"Check my account balance and transfer \$100."

Retrieving documents isn't enough.

The system needs to **perform actions**.

This leads us to tools.

❖ Chatbot 3: Tool-Augmented

Now we give the LLM access to tools.

For example:

LLM

- |— Calculator
- |— Search
- |— Weather API
- |— Database
- |— Custom API

Example

User:

"Convert \$100 to PKR."

The LLM can use a currency conversion tool.

```
graph TD
    User --> LLM1[LLM]
    LLM1 --> Tool[Currency Tool]
    Tool --> Rate[Exchange Rate]
    Rate --> LLM2[LLM]
    LLM2 --> Answer
```

❖ Tool Calling

The LLM doesn't necessarily execute the tool itself.

It produces a **tool call** specifying:

Tool: currency_converter

Arguments:

amount = 100

from = USD

to = PKR

The application executes it and returns the result.

❖ Why Tools Are Better

Now the AI can interact with the outside world.

It can:

- Calculate
- Search
- Query databases
- Call APIs
- Retrieve live information
- Perform actions

❖ Problems with Chatbot 3

There's still a major limitation.

Suppose we give the AI ten tools.

The user asks:

"Research the best laptop for machine learning under my budget, compare three options, and calculate the total cost."

Now multiple steps are required.

The system needs to determine:

What should I do first?

What tool should I use?

What should I do with the result?

Do I need another tool?

When am I finished?

A simple tool-enabled chatbot doesn't automatically provide a complete autonomous workflow.

❖ Chatbot 4: Agentic AI

Now we reach:

Agentic AI

An Agentic AI system can work toward a **goal** by making decisions and taking actions.

Definition

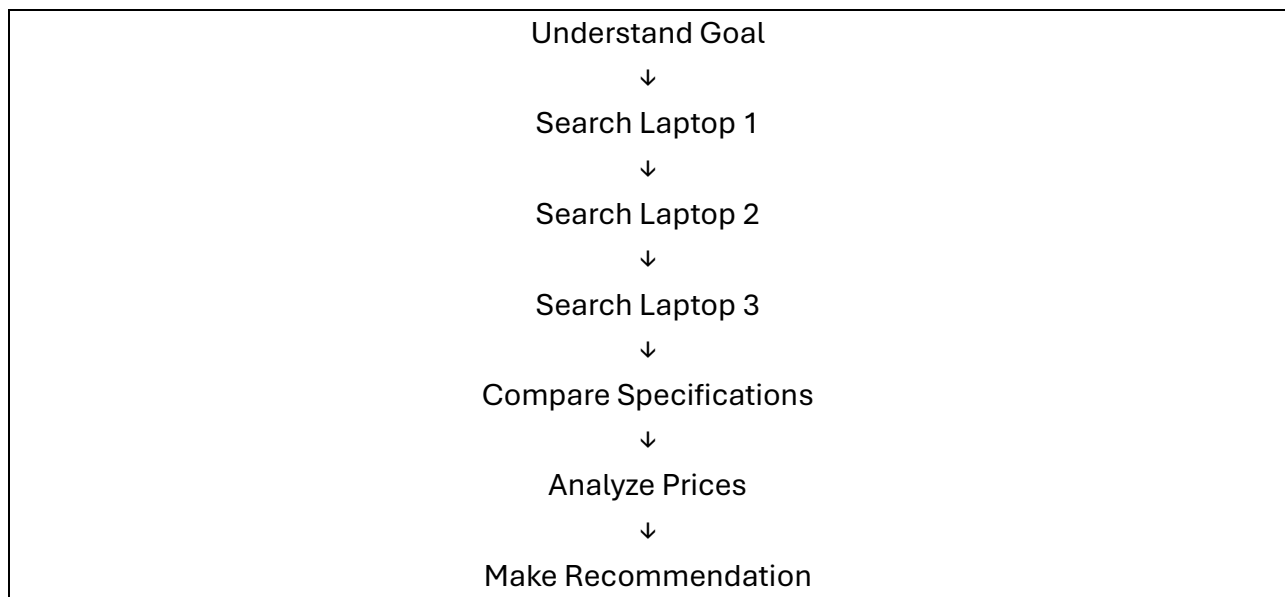
Agentic AI is AI that can decide what actions to take, use available tools, observe the results, and continue until it reaches a goal.

Example

User:

"Find the current price of a laptop, compare it with two alternatives, and tell me which one is best for machine learning."

An agent might do:



The developer doesn't necessarily hard-code every individual step.

The agent determines the next action based on the current situation.

❖ Generative AI vs Agentic AI

This is the **most important distinction** in this lesson.

Generative AI	Agentic AI
Generates content	Works toward goals
Responds to prompts	Takes actions
Usually one main generation step	Can involve multiple steps

Doesn't necessarily use tools	Can use tools
Limited decision-making	Makes decisions
Mostly reactive	Can be goal-directed

🔥 The Easiest Way to Remember

Generative AI asks:

"What should I say/create?"

Agentic AI asks:

"What should I do next to accomplish the goal?"

❖ Agentic AI Doesn't Mean "Human-Like AI"

This is an important clarification.

Agentic AI doesn't mean the AI is conscious or actually thinks like a human.

"Reasoning" in an agent usually means the system uses the model to determine:

- What information is needed
- Which action to take
- Which tool to call
- Whether another step is necessary

It's an engineering architecture, not proof of human-like consciousness.

❖ Problems with Agentic AI

Agentic systems are powerful, but they introduce new challenges.

✗ 1. Complexity

More components mean more things can fail.

✗ 2. Unpredictable Behavior

Because the agent makes decisions dynamically, its exact path can vary.

✗ 3. Tool Errors

An API or database might fail.

✗ 4. Infinite/Excessive Loops

An agent might continue taking actions without reaching a useful conclusion if the system isn't properly controlled.

✗ 5. Debugging

With a simple chain:

$A \rightarrow B \rightarrow C$

it's easy to see what happened.

With an agent:

$A \rightarrow B \rightarrow C$

→ D
↓
E
↗

the execution path can become much harder to understand.

❖ Conclusion

The video demonstrates an important evolution:

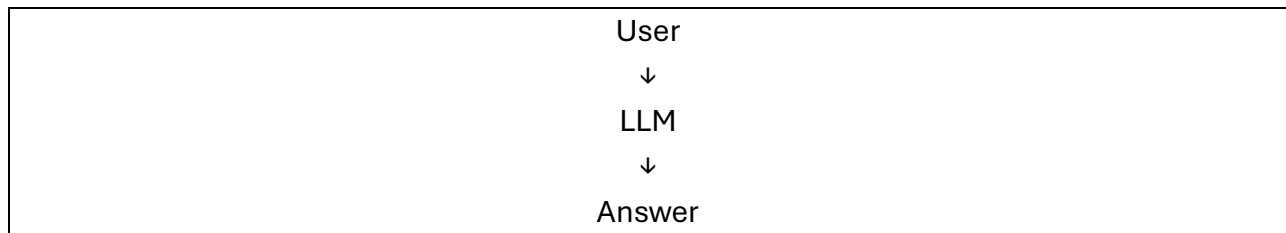


Each stage addresses limitations of the previous stage.

Complete Picture

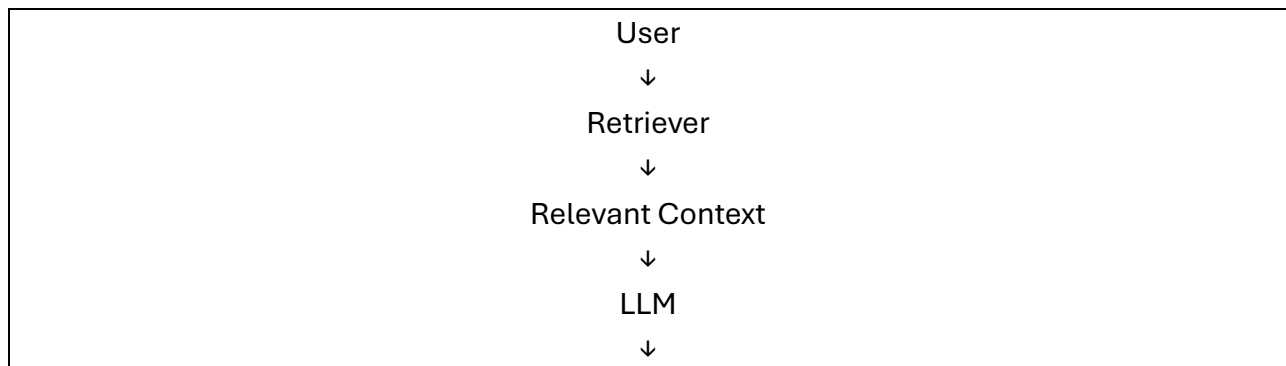
Let's put everything you've learned into one architecture.

Level 1 — Basic Generative AI



Purpose: Generate content.

Level 2 — RAG



Answer

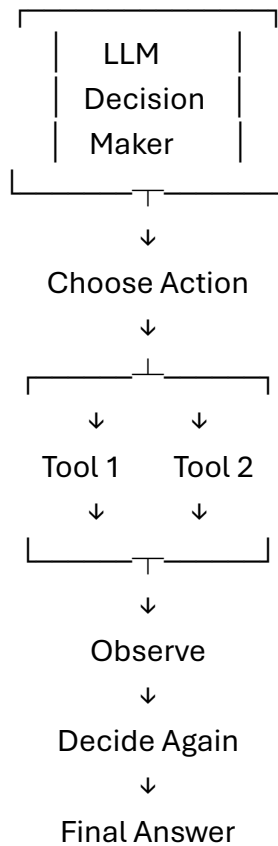
Purpose: Give the LLM access to external knowledge.

Level 3 — Tool-Augmented AI

User
↓
LLM
↓
Tool
↓
Result
↓
LLM
↓
Answer

Purpose: Give the LLM external capabilities.

Level 4 — Agentic AI



Purpose: Achieve a goal through multiple decisions and actions.

❖ **Where Does LangGraph Come In?**

This is where the title's **LangGraph** becomes important.

As agentic systems become complicated, simply writing:

LLM → Tool → LLM → Tool

can become difficult to manage.

LangGraph provides a way to represent the workflow as a **graph**.

In one sentence:

Generative AI generates content, RAG gives it external knowledge, tools give it capabilities, agents let it make decisions about those capabilities, and LangGraph helps you structure, control, and manage complex agentic workflows.